100
Pass

Mo Nayeem Bhati

PDF generated at: 10 Mar 2026 11:52:33 UTC

View this report on HackerRank

Score

100% • 175.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 61 min 9 sec on 10 Mar 2026 16:19:30 IST

Candidate Information

Email	nayeembhati96@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	10 Mar 2026 16:19:30 IST
Time taken	61 min 9 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	9 Mar 2026 19:28:56 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	100%	
3	Problem Solving Intermediate	100%	
4	Java Intermediate	100%	

Tags Distribution

Hard	100%	Spring Boot	100%
TDD	100%	Spring	100%
Application Properties Annotations	100%	Medium	100%
Implementation	100%	Arrays	100%
Algorithms	100%	Problem Solving	100%
Greedy	100%	Sorting	100%
Java	100%		

Questions

Spring Boot (Advanced) • 25 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Output Multiple Choice	Spring Boot (Advanced)	5/5	-	3 min 21 sec	✓

2	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 4 sec	✓
3	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 7 sec	✓
4	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 26 sec	✓
5	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 31 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Wind Farms Coding	Problem Solving (Intermediate)	75/75	-	30 min 52 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: DataVille's Optimal Storage Coding	Java (Intermediate)	75/75	-	20 min 7 sec	✓

1. Output

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }

   @Test
   public void test2()
   {
      logWriter.write("Entered Test 2");
      assertTrue(someClassToTest.otherMethod());
```

```
    }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ○ The output file is created but not deleted.
- ● **The output file is created and eventually deleted.** ✓
- ○ The create and delete blocks will not be executed.

No comments.

2. Configuration Properties

Correct

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
```

```
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [ ] The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- [ ] The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- [x] ** A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final. **
- [ ] An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.
- [ ] A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

3. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- ● **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- ○ None of the above

No comments.

---

4. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").
- [x] @Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

5. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Configuration class annotated with @EnableGlobalMethodSecurity
- Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)**
- None above

No comments.

6. Wind Farms

Correct

Coding | Implementation | Medium | Arrays | Algorithms | Problem Solving

Question description

An energy company needs to place a control center to manage a distributed wind farm. Your task is to determine the optimal location for the control center that minimizes the total connection cost.

For each wind farm *i*:

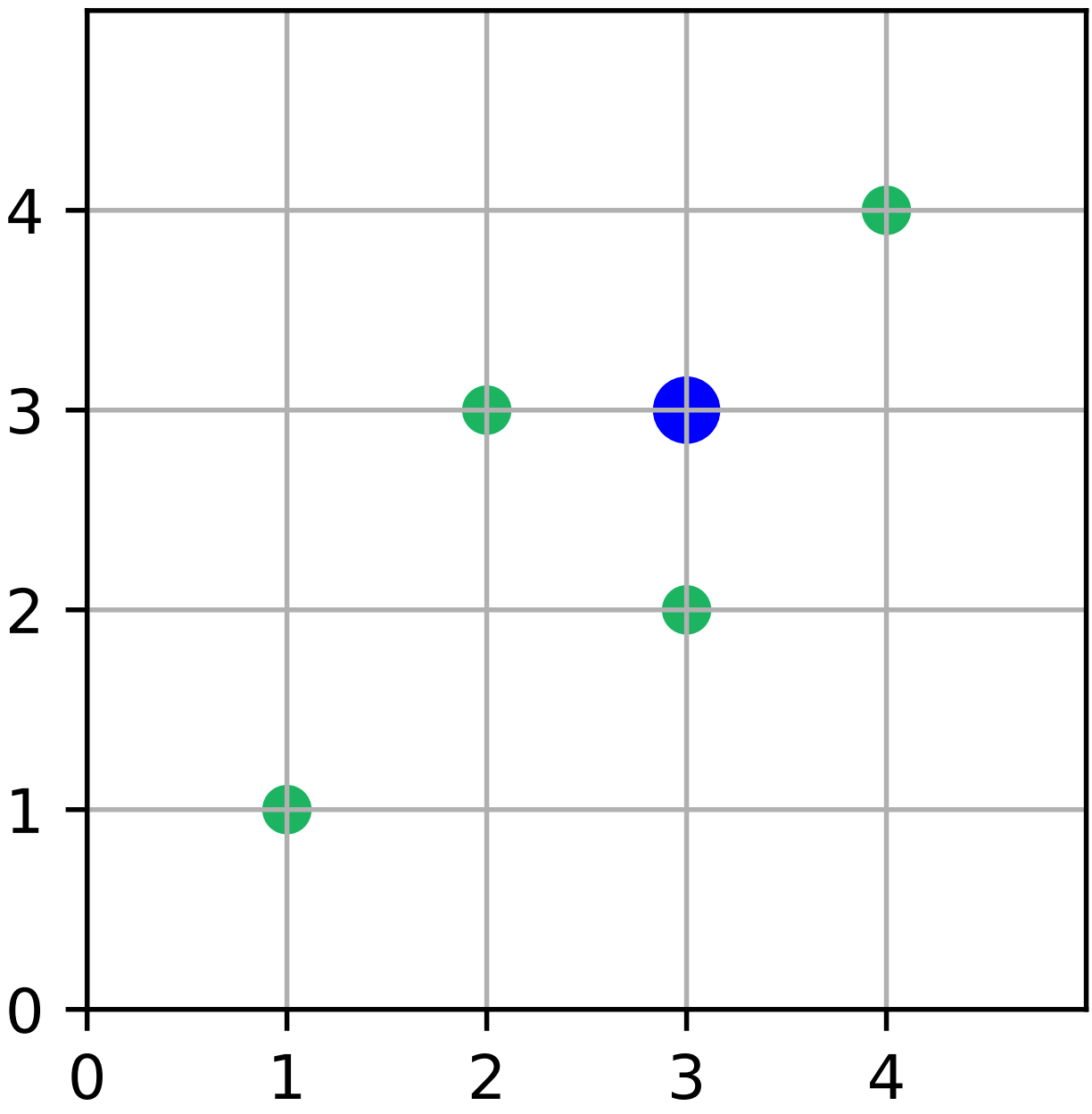
- The farm is located at coordinates *(x[i], y[i])*
- The control center is at coordinates *(cx, cy)*
- The distance is calculated as *|x[i] - cx| + |y[i] - cy|*
- The connection cost is *premium[i] * distance*

Example

x = [1, 3, 2, 4]

y = [1, 2, 3, 4]

premium = [1, 3, 2, 4]



The optimal location is at position [3, 3], resulting in:

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 3| = 4	1	4 * 1 = 4
1	(3, 2)	|3 - 3| + |2 - 3| = 1	3	1 * 3 = 3
2	(2, 3)	|2 - 3| + |3 - 3| = 1	2	1 * 2 = 2
3	(4, 4)	|4 - 3| + |4 - 3| = 2	4	2 * 4 = 8

The total connection cost is 17, which is the minimum possible.

Function Description

Complete the function *windFarms* in the editor with the following parameter(s):

int premium[n]: the distance multiplier for the cost to connect
int x[n]: *x* coordinates of the wind farms
int y[n]: *y* coordinates of the wind farms

Returns

int: the minimum cost to connect the wind farms

Constraints

- $1 \le n \le 10^3$
- $1 \le x[i], y[i] \le 10^4$
- $1 \le premium[i] \le 50$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the size of the array *premium*.
The next *n* lines each contain an element, *premium[i],* where 0 ≤ i < n.
The next line contains an integer *n*, the size of *x[]*.
The next *n* lines each contain an element, *x[i]*.
The next line contains an integer *n*, the size of *y[]*.
The next *n* lines each contain an element, *y[i]*.

Note: *n* is the same value in each instance.

▼ SAMPLE CASE 0

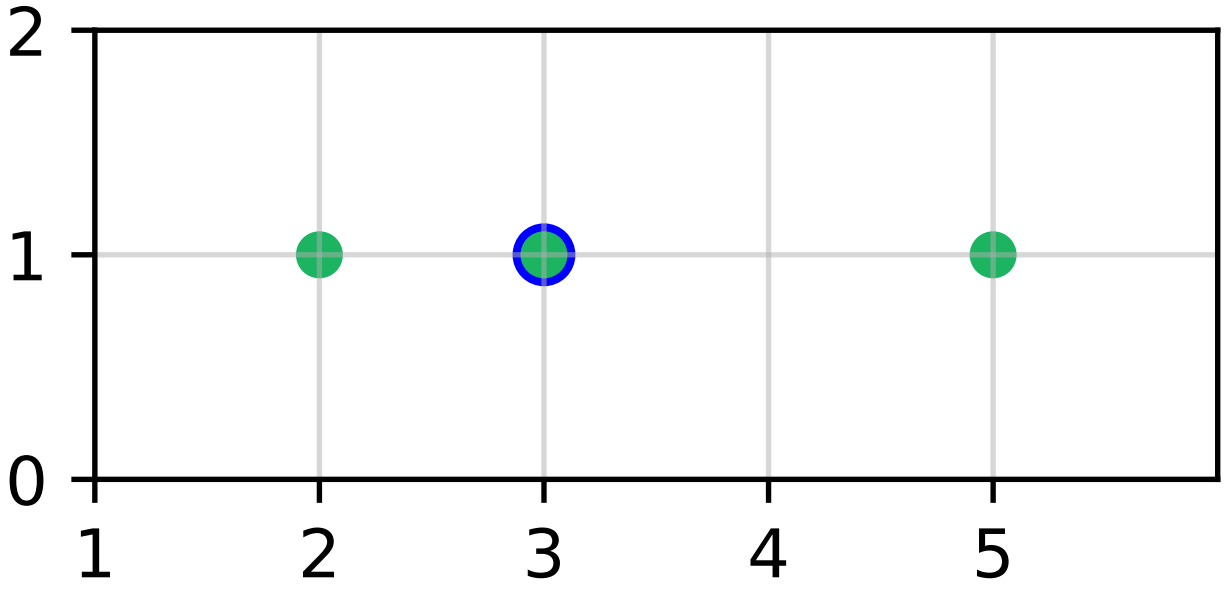
Sample Input 0

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
2    →  x = [2, 3, 5]
3
5
3    →  y[] size n = 3
1    →  y = [1, 1, 1]
1
1
```

Sample Output 0

```
3
```

Explanation 0



Locate the controller at farm 1.

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 1| = 1	1	1 * 1 = 1
1	(3, 1)	|3 - 3| + |1 - 1| = 0	1	0 * 1 = 0

2	(5, 1)	|5 - 3| + |1 - 1| = 2	1	2 * 1 = 2

▼ SAMPLE CASE 1

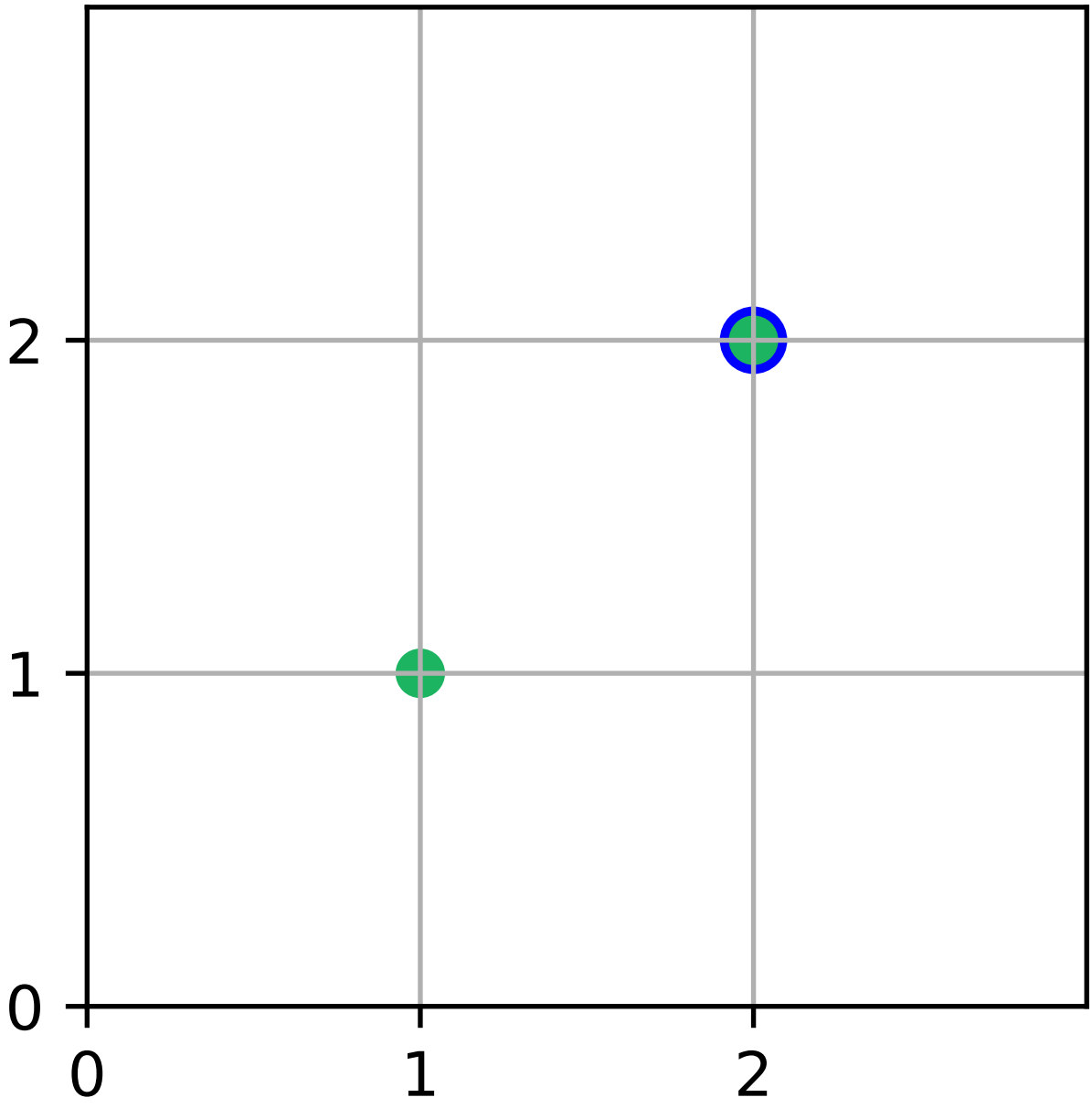
Sample Input 1

```
STDIN    Function
-----    --------
2    →  premium[] size n = 2
1    →  premium = [1, 10]
10
2    →  x[] size n = 2
1    →  x = [1, 2]
2
2    →  y[] size n = 2
1    →  y = [1, 2]
2
```

Sample Output 1

```
2
```

Explanation 1



The sum of the distances to a center at (1, 1), (1, 2), (2, 2), or (2, 1) is 2. Since the premium to connect farm 1 is 10, it is best to locate at position (2, 2) for a cost of 2 * 1 + 0 * 10 = 2.

▼ SAMPLE CASE 2

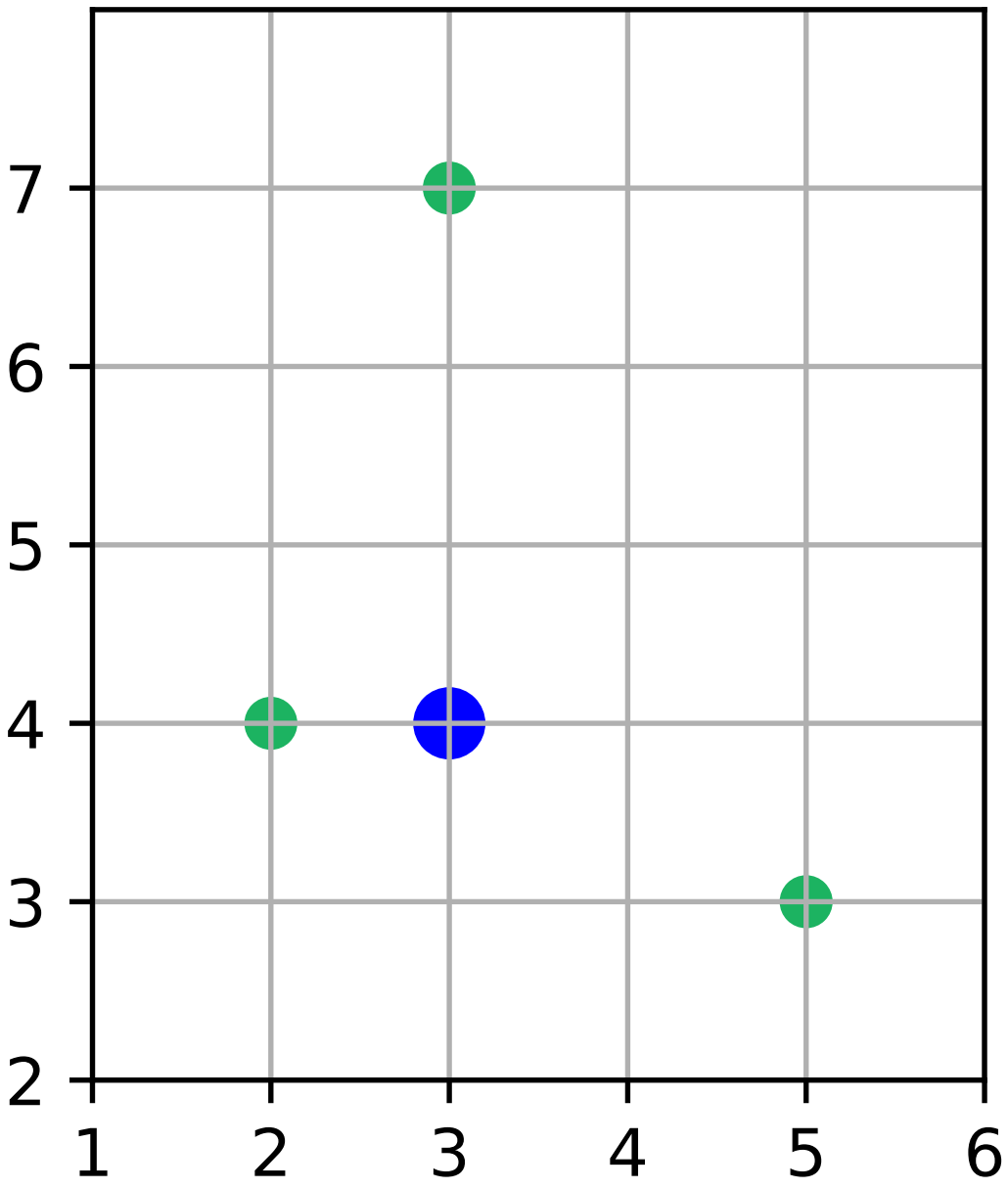
Sample Input 2

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
5    →  x = [5, 2, 3]
2
3
3    →  y[] size n = 3
3    →  y = [3, 4, 7]
4
7
```

Sample Output 2

```
7
```

Explanation 2



The optimal location is (3, 4) for a cost of 3 + 1 + 3 = 7.

Interviewer guidelines

▼ SOLUTION

Skills: Loops and counters, I/O

The cost can be calculated for x and y coordinates separately. The function examines each possible x and y coordinate for the control center separately (from 1 to 10000), computes the total cost of connecting all farms for each possible coordinate, and captures the minimum cost found. The ultimate minimum cost is the summation of these minimum costs for x and y coordinates separately.

Optimal Solution

```
def windFarms(premium, x, y):
    minCostX = 10**10
    n = len(x)

    for cx in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cx - x[i])

        minCostX = min(minCostX, cost)

    minCostY = 10**10
    for cy in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cy - y[i])

        minCostY = min(minCostY, cost)

    return minCostX + minCostY
```

▼ **COMPLEXITY ANALYSIS**

Time Complexity - O(n)

The universe of x and y values are tested against 10000 values.

Space Complexity - O(1)

Constant space is required regardless of n.

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'windFarms' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY premium
     *  2. INTEGER_ARRAY x
     *  3. INTEGER_ARRAY y
     */

    public static int windFarms(List<Integer> premium, List<Integer> x, List<Integer> y) {
    // Write your code here
        int n = premium.size();

        int X = findM(x, premium);
        int Y = findM(y, premium);

        long cost = 0;

        for(int i=0; i< n; i++){
            cost += (long) premium.get(i) * (Math.abs(x.get(i) - X) + Math.abs(y.get(i) - Y));
        }

        return (int)cost;
    }
    private static int findM(List<Integer> cordinates, List<Integer> premium){
```

```java

        int n = cordinates.size();
        List<int[]> list = new ArrayList<>();

        for(int i=0; i<n; i++){
            list.add(new int[]{cordinates.get(i), premium.get(i)});
        }

        list.sort(Comparator.comparingInt(a -> a[0]));

        long total =0;
        for(int w : premium){
            total +=w;
        }

        long half = (total+1)/2;
        long prefix =0;

        for(int[] p : list){
            prefix +=p[1];
            if(prefix >= half){
                return p[0];
            }
        }
        return 0;
    }


}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int premiumCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> premium = IntStream.range(0, premiumCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
```

```
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int xCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> x = IntStream.range(0, xCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int yCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> y = IntStream.range(0, yCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.windFarms(premium, x, y);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

TestCase 0	Easy	Sample	Success	1	0.058 sec	37.6 KB
TestCase 1	Easy	Sample	Success	1	0.0599 sec	38.1 KB
TestCase 2	Easy	Sample	Success	1	0.053 sec	37.6 KB
TestCase 3	Easy	Sample	Success	2	0.1058 sec	37.7 KB
TestCase 4	Easy	Hidden	Success	2	0.0576 sec	37.4 KB
TestCase 5	Easy	Hidden	Success	2	0.0582 sec	37.9 KB
TestCase 6	Medium	Sample	Success	6	0.0773 sec	38.2 KB
TestCase 7	Medium	Hidden	Success	6	0.0855 sec	38.1 KB
TestCase 8	Medium	Hidden	Success	6	0.0683 sec	38.1 KB
TestCase 9	Hard	Hidden	Success	12	0.1138 sec	43.2 KB
TestCase 10	Hard	Hidden	Success	12	0.0759 sec	44.2 KB
TestCase 11	Hard	Hidden	Success	12	0.1087 sec	43.5 KB
TestCase 12	Hard	Hidden	Success	12	0.0848 sec	44.1 KB

No comments.

7. Java: DataVille's Optimal Storage

Correct

Coding Greedy Sorting Medium Java

Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

Function Description

Complete the function g*etOptimalStorage* in the editor below.

Function Parameters

int databaseStorage[n]: the current organization of the database's storage, with 0s for standard data and 1s for priority data

Returns

int: the minimum number of operations required to minimize transitions between 0s and 1s

Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in \{0, 1\}$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.
Each of the next lines contains an integer, *databaseStorage[i].*

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
0
1
```

Sample Output 0

```
1
```

Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
```

```
1
0
0
1
1
0
0
0
1
```

databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]

Sample Output 1

```
2
```

Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;
```

```
        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }

        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

Time Complexity - O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

Space Complexity - O(1)

We use only 2 variables, hence the space complexity is O(1).

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
```

```java
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
         int n = databaseStorage.size();
         int count01=0;
         int count10=0;
         int c0=0;

         if(databaseStorage.size() <= 1)
         return 0;

         for(int bit : databaseStorage){
            if(bit ==0){
                c0++;
            }
         }

         for(int i=0; i<c0; i++){
                if(databaseStorage.get(i) == 1){
                    count01++;
                }
         }
         int c1=n-c0;
         for(int i=0; i<c1; i++){
                if(databaseStorage.get(i) == 0){
                    count10++;
                }
         }
        return Math.min(count01, count10);
    }
```

```java

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0564 sec	37.4 KB

TestCase 1	Easy	Sample	Success	1	0.0714 sec	37.4 KB
TestCase 2	Easy	Sample	Success	1	0.0568 sec	37.6 KB
TestCase 3	Easy	Hidden	Success	3	0.0467 sec	37.6 KB
TestCase 4	Easy	Hidden	Success	3	0.0534 sec	37.3 KB
TestCase 5	Easy	Hidden	Success	3	0.0485 sec	37.5 KB
TestCase 6	Easy	Hidden	Success	4	0.0508 sec	37.6 KB
TestCase 7	Easy	Hidden	Success	4	0.0537 sec	37.6 KB
TestCase 8	Easy	Hidden	Success	5	0.0957 sec	37.3 KB
TestCase 9	Easy	Hidden	Success	5	0.0659 sec	37.3 KB
TestCase 10	Easy	Hidden	Success	5	0.066 sec	37.3 KB
TestCase 11	Medium	Hidden	Success	7	0.073 sec	41.2 KB
TestCase 12	Medium	Hidden	Success	8	0.0623 sec	39.4 KB
TestCase 13	Hard	Hidden	Success	10	0.0765 sec	39.8 KB

TestCase 14	Hard	Hidden	Success	15	0.1715 sec	68.3 KB

No comments.